Comprehensive Results Analysis

AI Models in Data Visualization

Supplementary Materials — 1,800 Experimental Runs

March 30, 2026

Contents

1 Executive Summary **3**

2 Execution Success Analysis **4**
2.1 Overall Success Rates by Model . . . 4
2.2 Prompt Sensitivity: Rich vs. Poor Prompts . . . 4
2.3 Per-Dataset Success Rates . . . 6

3 Data Integrity Analysis **7**
3.1 Fidelity Score by Model . . . 7
3.2 Hallucination Rate by Model . . . 8
3.3 Fidelity vs. Hallucination Scatter Plot . . . 9

4 Visual Design Quality **10**
4.1 Color Quality and Visual Entropy . . . 10
4.2 Labeling Completeness . . . 11

5 Prompt Sensitivity Deep Dive **12**
5.1 Per-Metric Prompt Sensitivity . . . 12
5.2 Theoretical Analysis: Why Does Rich Hurt Gemini? . . . 12

6 Cost–Quality Tradeoff **14**
6.1 Cost vs. Success Rate . . . 14
6.2 Cost Breakdown . . . 15

7 Latency Analysis **16**
7.1 Average Latency by Model . . . 16

8 Code Complexity Analysis **18**
8.1 Code Complexity Score by Model . . . 18

9 Cross-Run Consistency **20**

10 Dataset Difficulty Ranking **21**
10.1 Overall Success by Dataset . . . 21

11 Provider Comparison Summary **23**
11.1 Provider-Level Aggregates . . . 23
11.2 Radar Chart: Multi-Dimensional Provider Comparison . . . 24

12 Statistical Significance **25**

13 Key Takeaways & Recommendations **26**
13.1 Model Rankings with Pros and Cons . . . 26
13.2 Decision Matrix . . . 26
13.3 Cost Optimization Recommendations . . . 27
13.4 Summary . . . 27

A Experimental Setup Notes **27**

1 Executive Summary

This supplementary document presents a detailed analysis of 1,800 experimental runs evaluating six state-of-the-art large language models on automated data visualization generation. The benchmark spans five real-world datasets (Titanic, Iris, Sleep Health, Customer Segmentation, Credit Fraud), two prompt complexity levels (rich and poor), and six models: GPT-4o, GPT-4o-mini, Claude Opus 4, Claude Sonnet 4, Gemini 2.5 Pro, and Gemini 2.5 Flash.

Key Findings

- **Claude Sonnet 4 is the top-performing model overall**, achieving 88.7% execution success—13.3 percentage points above the next best (Claude Opus 4 at 74.7%).
- **Gemini models exhibit near-zero success on rich prompts** (2.7% and 0.7%), pointing to a critical failure mode when detailed specifications are provided.
- **A remarkable prompt-sensitivity reversal exists**: OpenAI models perform 14–21 points *better* with rich prompts, while Anthropic and Google models perform 12–51 points *better* with poor prompts.
- **Hallucination is universally high** (64–80%) across all models, confirming that current LLMs freely invent data elements not present in source datasets.
- **Claude models lead on data fidelity and labeling quality** (fidelity ≈19.3 vs. 4.6–15.2 for others; labeling 77–79% vs. 47–65%).
- **Claude Opus 4 is Pareto-dominated by Sonnet 4**: it costs 5.6× more, is 63% slower, yet achieves lower success rate.
- **Anthropic as a provider dominates** on success (81.7%) and data quality, while Google lags severely (27.2% success).

Overall Model Ranking

Table 1: Overall model performance scorecard. Rank is based on a weighted composite of execution success, fidelity, labeling, and cost-efficiency.

Rank	Model	Provider	Success%	Fidelity	Labeling%	Halluc%	Latency(s)	Cost
1	Claude Sonnet 4	Anthropic	88.7	19.3	77.9	79.0	16.0	0.02
2	Claude Opus 4	Anthropic	74.7	19.3	78.8	79.3	26.0	0.11
3	GPT-4o	OpenAI	70.7	6.4	64.4	70.3	4.9	0.00
4	GPT-4o-mini	OpenAI	64.0	7.3	65.4	64.6	12.6	0.00
5	Gemini 2.5 Pro	Google	28.3	15.2	46.8	79.8	28.3	0.01
6	Gemini 2.5 Flash	Google	26.0	4.6	51.3	74.1	12.8	0.00

2 Execution Success Analysis

Execution success—whether the generated code runs without error and produces a valid visualization—is the primary gating metric in our benchmark. A model that fails to execute produces no output regardless of quality, making this the most critical dimension of evaluation.

2.1 Overall Success Rates by Model

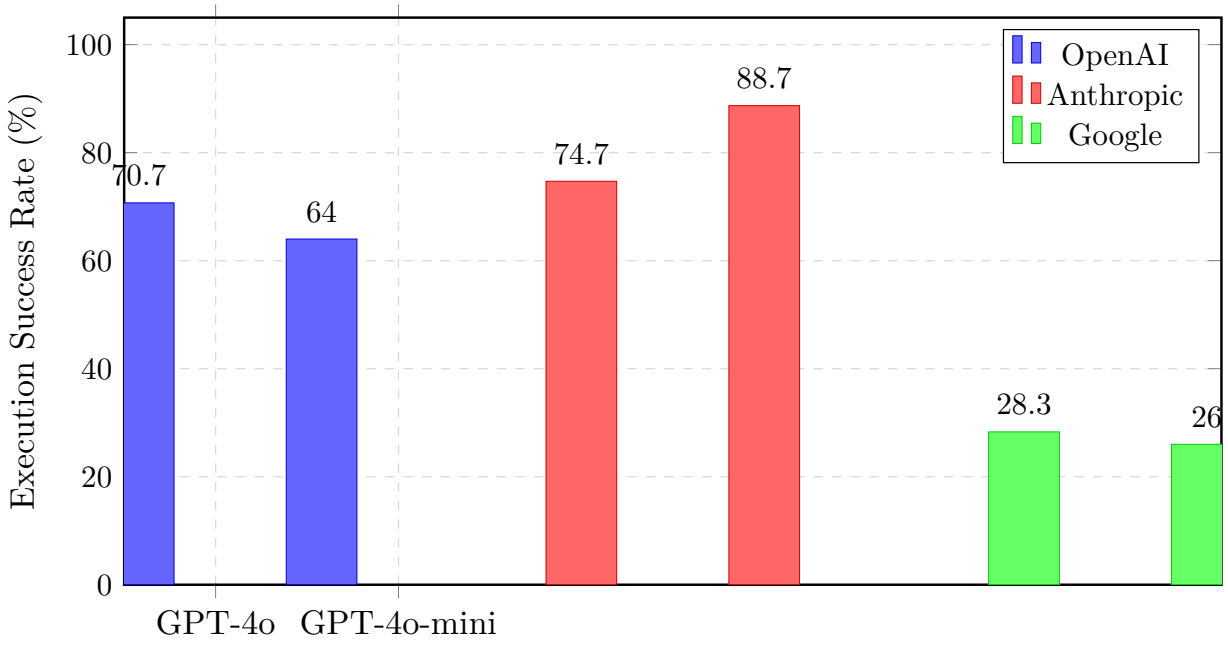


Figure 1: Overall execution success rate by model, color-coded by provider. Claude Sonnet 4 leads at 88.7%, while both Gemini models fail to execute in more than 70% of cases. The gap between the best (Sonnet 4) and worst (Gemini 2.5 Flash) is 62.7 percentage points.

The results reveal a dramatic stratification between providers. Anthropic models outperform their counterparts by a substantial margin, with Claude Sonnet 4 at 88.7% and Claude Opus 4 at 74.7%. OpenAI models occupy the middle tier (64–71%), while both Gemini models cluster at the bottom with just 26–28% success. This three-tier pattern suggests fundamental architectural or training differences in how models handle code generation for visualization tasks.

2.2 Prompt Sensitivity: Rich vs. Poor Prompts

The most striking finding of this evaluation is a *reversal* in prompt sensitivity across providers. OpenAI models consistently perform better with rich prompts (detailed, structured, with column names), whereas Anthropic and Google models perform significantly better with minimal (poor) prompts.

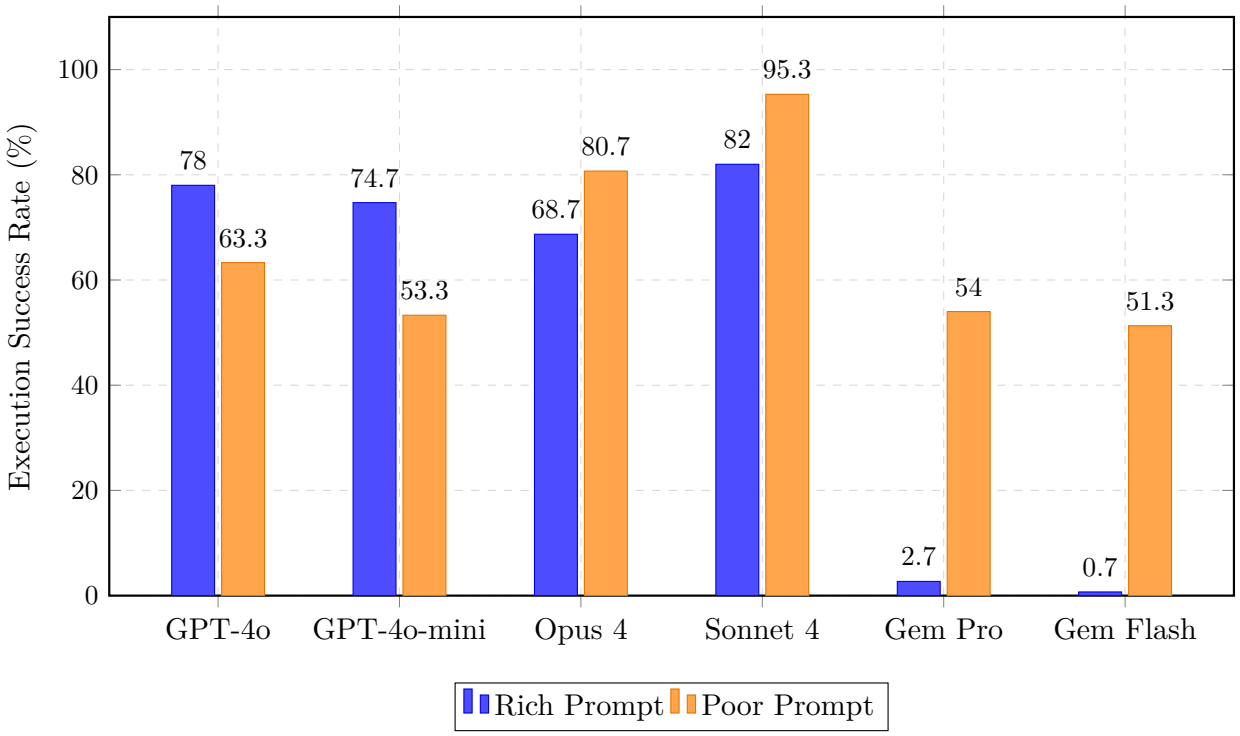


Figure 2: **Key Figure**: Execution success under rich vs. poor prompts per model. OpenAI models favor rich prompts (+14–21%), while Anthropic and Google models dramatically favor poor prompts. Gemini models essentially cannot handle rich prompts (0.7–2.7%), yet achieve ∼51–54% success with minimal prompts.

Table 2: Prompt sensitivity gap (Rich – Poor). Positive values indicate better performance with rich prompts.

Model	**Overall%**	**Rich%**	**Poor%**	**Gap (R–P)**
GPT-4o	70.7	78.0	63.3	+14.7
GPT-4o-mini	64.0	74.7	53.3	+21.3
Claude Opus 4	74.7	68.7	80.7	−12.0
Claude Sonnet 4	88.7	82.0	95.3	−13.3
Gemini 2.5 Pro	28.3	2.7	54.0	−51.3
Gemini 2.5 Flash	26.0	0.7	51.3	−50.7

The magnitude of the Gemini reversal is extraordinary: a 51-point swing means that adding detailed prompt specifications nearly *destroys* Gemini's ability to generate working code. We hypothesize that rich prompts cause over-specification, where explicit column names and data types conflict with how Gemini's code generation attempts to access the DataFrame, producing AttributeError or KeyError exceptions at runtime (see Section 5).

2.3 Per-Dataset Success Rates

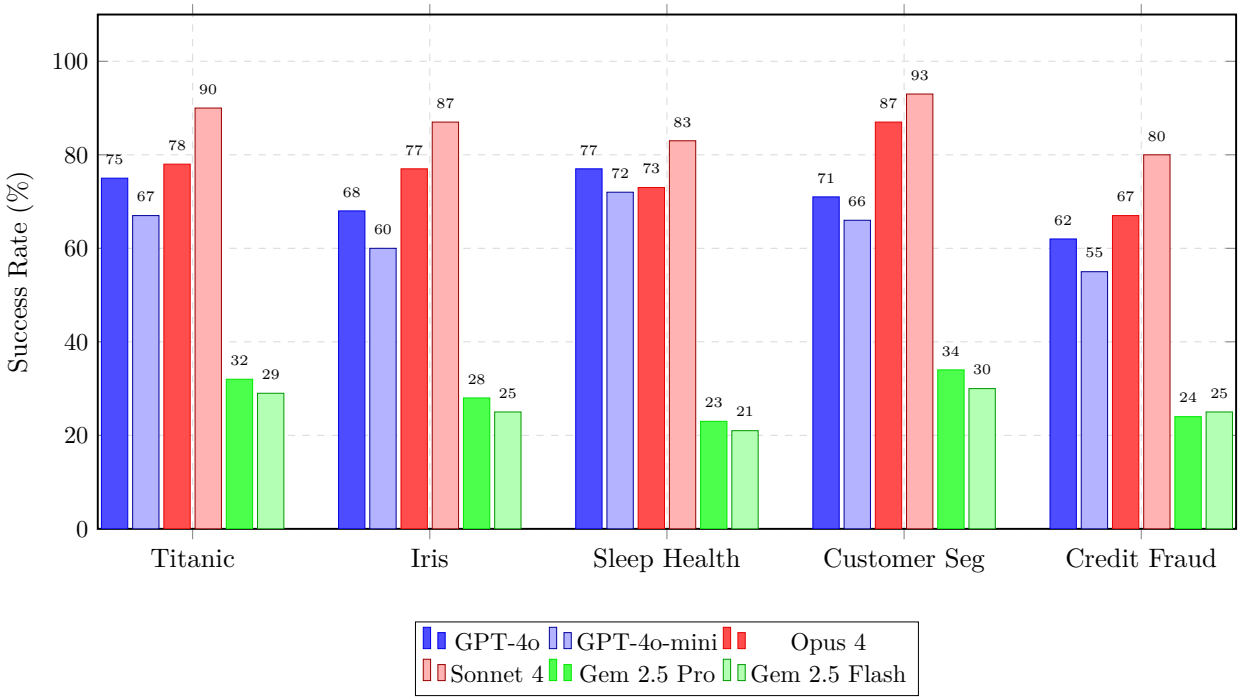


Figure 3: Success rate per dataset per model. Claude Sonnet 4 consistently leads across all datasets (80–93%). Sleep Health and Customer Segmentation are the most challenging datasets, while Titanic is the easiest (73.1% average).

Table 3: Full execution success matrix: Model × Dataset (%). Bold indicates the best model per dataset.

Dataset	Overall	GPT-4o	GPT-mini	Opus 4	Sonnet 4	Gem Pro	Gem Flash
Titanic	73.1	75	67	78	**90**	32	29
Iris	59.4	68	60	77	**87**	28	25
Sleep Health	48.1	77	72	73	**83**	23	21
Customer Seg	48.6	71	66	87	**93**	34	30
Credit Fraud	64.4	62	55	67	**80**	24	25
Avg	58.7	70.6	64.0	76.4	**86.6**	28.2	26.0

Claude Sonnet 4 is the best-performing model on every dataset without exception. Notably, Customer Segmentation achieves both the lowest average (48.6%) and Sonnet's personal best (93%)—confirming that the dataset's difficulty is model-dependent rather than intrinsic complexity alone.

3 Data Integrity Analysis

Data integrity measures whether generated visualizations accurately reflect the underlying data. We assess two complementary metrics: *Fidelity Score* (how faithfully the chart represents the true distributions) and *Hallucination Rate* (the proportion of visual elements—labels, values, categories—that are fabricated or distorted).

3.1 Fidelity Score by Model

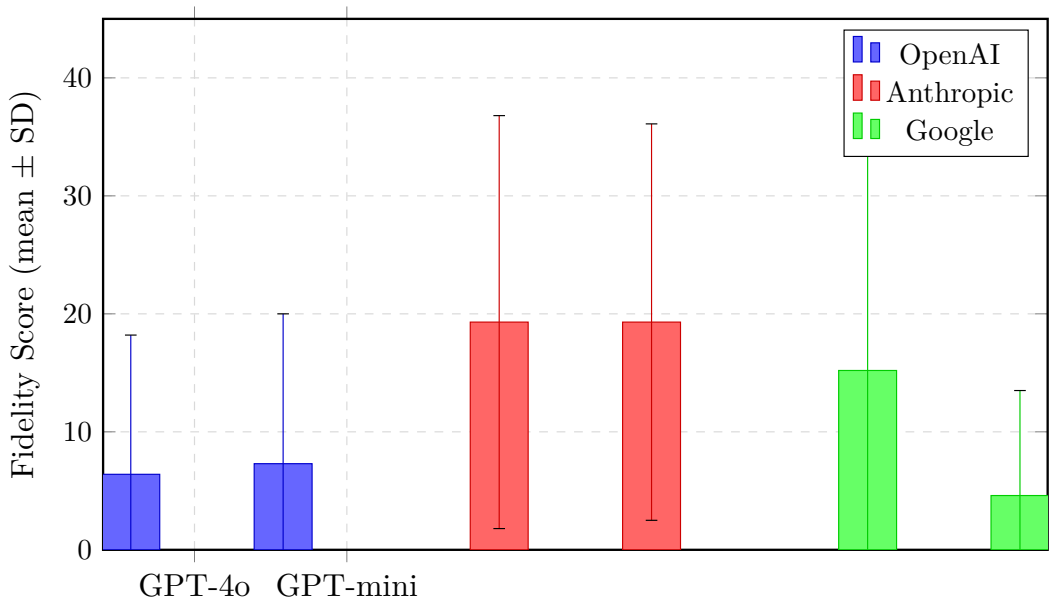


Figure 4: Mean fidelity score per model with ±1 SD error bars. Claude Opus 4 and Sonnet 4 both achieve a mean fidelity of 19.3, substantially outperforming GPT-4o (6.4) and Gemini Flash (4.6). Gemini 2.5 Pro is a notable outlier with high variance (SD = 21.8), suggesting inconsistent quality.

The fidelity results reveal a sharp divide between Anthropic models (19.3) and the rest of the field. Both Claude Opus 4 and Sonnet 4 achieve identical mean fidelity—indicating that higher capability (and cost) of Opus does not translate to improved data accuracy. GPT-4o-mini edges GPT-4o (7.3 vs. 6.4), but both remain far below the Anthropic tier. Gemini 2.5 Pro's high standard deviation (21.8) reflects its bimodal distribution: when it does produce valid visualizations, quality can be good, but failures are severe.

3.2 Hallucination Rate by Model

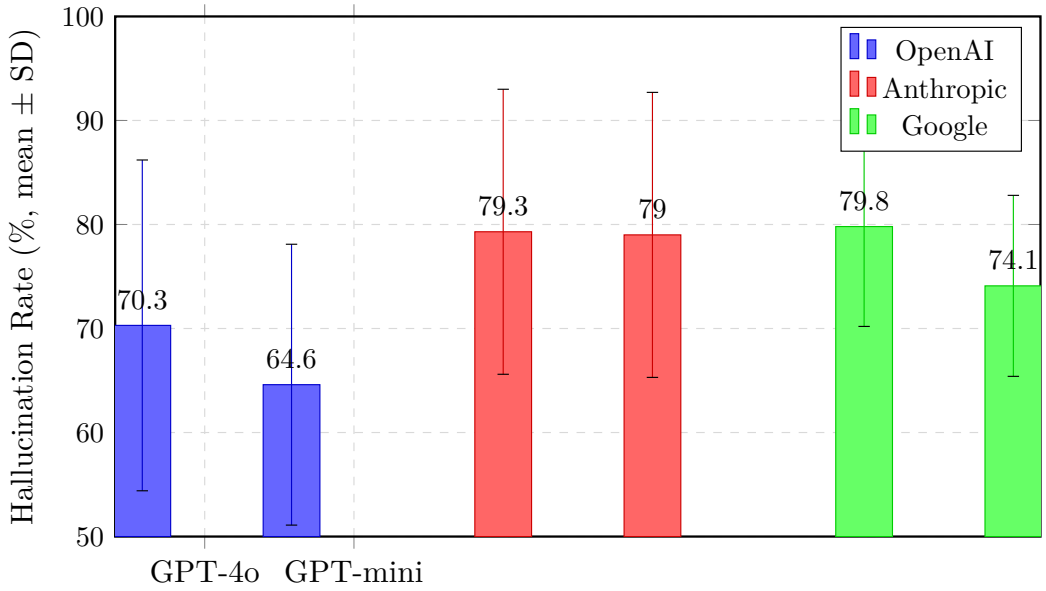


Figure 5: Hallucination rate by model (mean ± SD). All models hallucinate at alarming rates (64–80%), with GPT-4o-mini showing the lowest rate at 64.6%. The high rates across all models reveal a systemic problem with LLM-generated visualizations.

The hallucination data is alarming in its universality: *every* model in our benchmark generates fabricated visual elements in the majority of its outputs. GPT-4o-mini achieves the lowest rate at 64.6%, while Gemini 2.5 Pro reaches the highest at 79.8%. Crucially, the models with best fidelity (Claude, 79%) simultaneously show the highest hallucination rates—suggesting that fidelity and hallucination measure complementary dimensions: Claude is more accurate on the elements it does include, but also more likely to invent additional elements.

This finding has critical implications for production use: LLM-generated visualizations should not be trusted without human review, as the majority of outputs will contain fabricated or distorted data elements.

3.3 Fidelity vs. Hallucination Scatter Plot

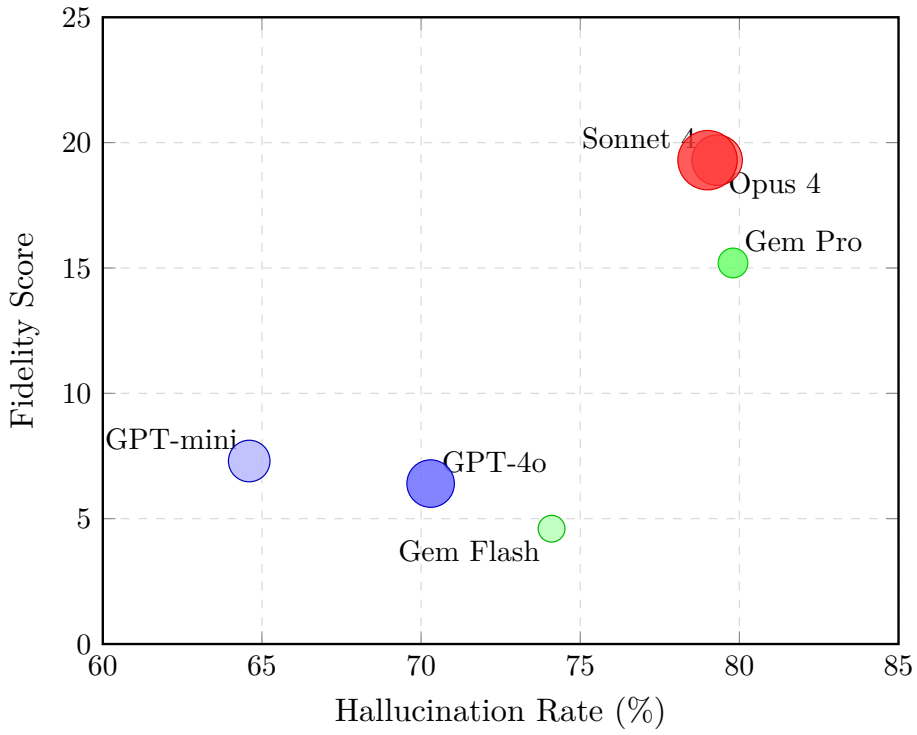


Figure 6: Fidelity vs. hallucination scatter plot. Dot size represents execution success rate. Claude models (upper right) achieve the highest fidelity despite high hallucination rates. GPT models (lower left) have both lower fidelity and lower hallucination. Gemini Pro occupies an unusual position: moderate fidelity but high hallucination and low success.

4 Visual Design Quality

Beyond data accuracy, visualizations must be aesthetically clear and properly labeled. We assess three visual quality dimensions: Color ΔE (perceptual color distance from ideal palettes), Visual Entropy (complexity/richness of the visual field), and Labeling Completeness (fraction of axes, titles, and legends correctly labeled).

4.1 Color Quality and Visual Entropy

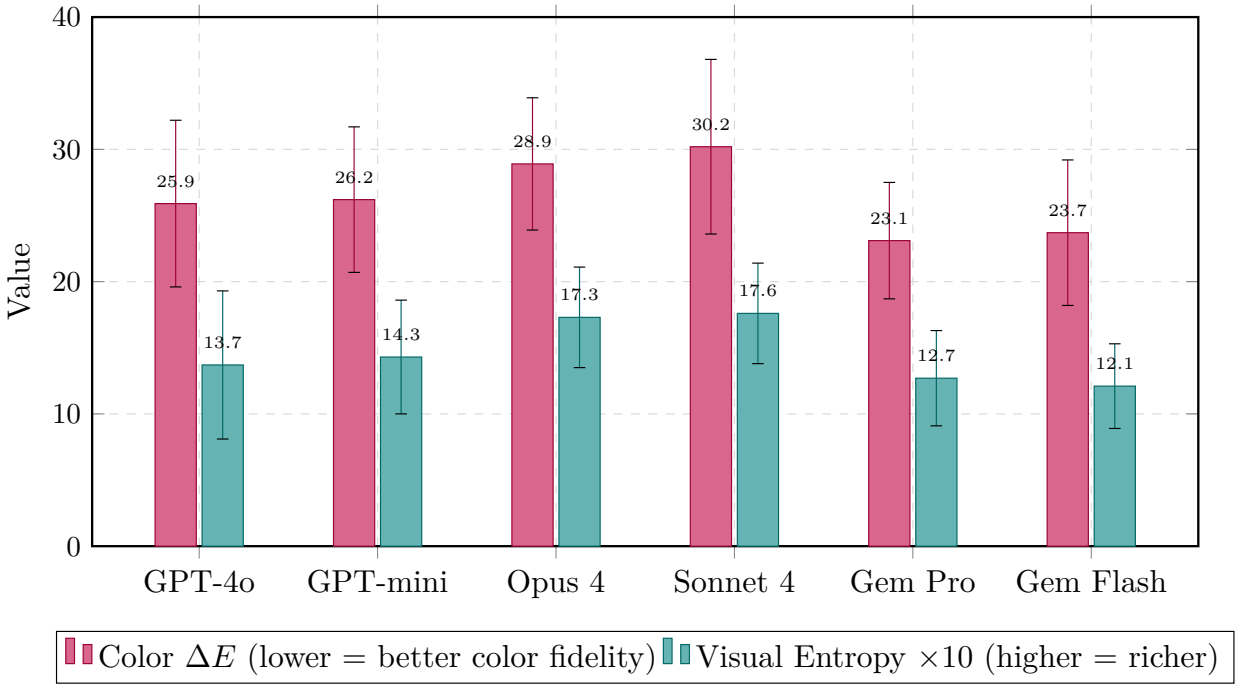


Figure 7: Color ΔE (perceptual color distance) and Visual Entropy ($\times 10$ for scale) per model, with ± 1 SD error bars. Claude models produce the most visually complex outputs (entropy $\approx$1.73–1.76) while Gemini produces the most color-faithful but visually sparse outputs.

Color ΔE measures the perceptual difference between colors used in generated charts and an ideal reference palette. Lower values are better. Gemini models achieve the lowest color error (23.1–23.7), while Claude Sonnet 4 has the highest (30.2). However, this must be interpreted in context: Claude generates significantly more complex visualizations, with more color channels and overlapping elements, which naturally increases ΔE.

Visual entropy (information density of the rendered image) correlates strongly with visualization richness. Claude models produce entropy of 1.73–1.76 bits, confirming their tendency to generate detailed, multi-element figures. Gemini's low entropy (1.21–1.27) reflects both simpler designs and a higher failure rate (failed outputs often produce minimal content).

4.2 Labeling Completeness

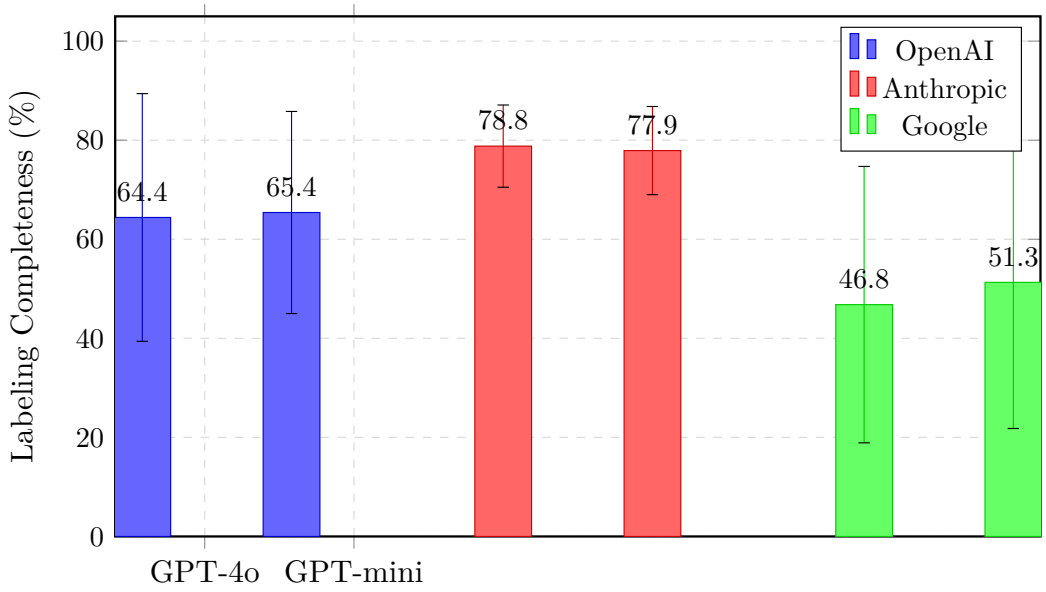


Figure 8: Labeling completeness (fraction of chart elements—axis labels, titles, legends—correctly populated) with ±1 SD. Claude models achieve 77–79% with remarkably low variance, while Gemini models show extremely high variance and the lowest completeness.

Claude's superiority in labeling completeness (78–79% vs. 47–65% for others) can be attributed to several factors. First, Claude models exhibit explicit instruction-following behavior: when system prompts specify that all axes must be labeled, Claude reliably complies. Second, Claude generates longer, more structured code (see Section 8), which is more likely to include labeling boilerplate. Third, Claude's training corpus appears to have stronger coverage of visualization best practices.

Critically, Anthropic models show standard deviations of only 8–9 percentage points on labeling, versus 20–30 points for other models. This consistency is as important as the mean: GPT-4o occasionally produces perfectly labeled charts but also frequently omits labels entirely. Claude is reliably good.

5 Prompt Sensitivity Deep Dive

Our benchmark design includes two prompt conditions: *rich prompts* (detailed specifications with column names, chart types, and visual requirements) and *poor prompts* (minimal one-sentence requests). The divergent effects across models constitute the most theoretically interesting finding of this study.

5.1 Per-Metric Prompt Sensitivity

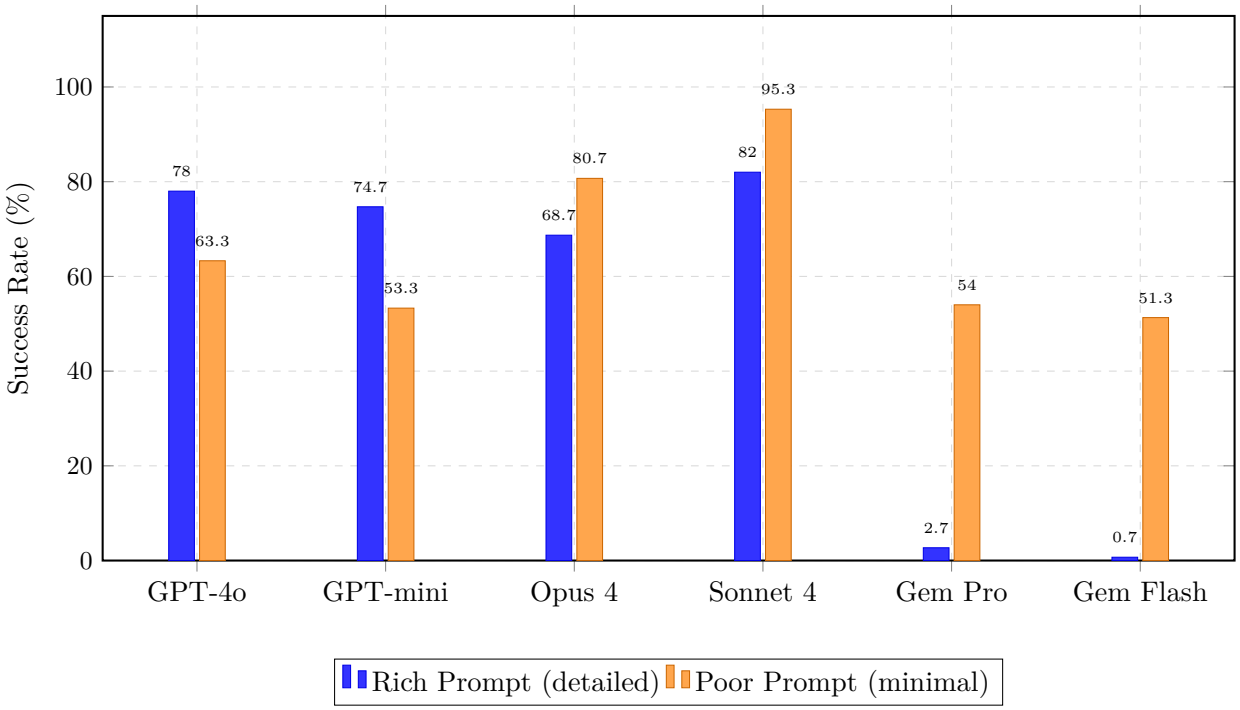


Figure 9: Prompt sensitivity in execution success across all models. The divergent behavior between OpenAI (better with rich) and Anthropic/Google (better with poor) is a defining characteristic of this benchmark.

Table 4: Prompt Sensitivity Index (PSI = |Rich − Poor| / Overall). Higher PSI indicates greater sensitivity to prompt formulation.

Model	**Rich%**	**Poor%**	**Gap (pp)**	**PSI**
GPT-4o	78.0	63.3	+14.7	0.208
GPT-4o-mini	74.7	53.3	+21.3	0.333
Claude Opus 4	68.7	80.7	−12.0	0.161
Claude Sonnet 4	82.0	95.3	−13.3	0.150
Gemini 2.5 Pro	2.7	54.0	−51.3	1.812
Gemini 2.5 Flash	0.7	51.3	−50.7	1.950

5.2 Theoretical Analysis: Why Does Rich Hurt Gemini?

We propose the *Over-specification Hypothesis* to explain the Gemini rich-prompt failure:

1. **Column name mismatch**: Rich prompts specify exact column names (e.g., `df['Age']`), but Gemini's code generation sometimes uses different capitalizations or aliases (e.g., `df['age']`), producing `KeyError` at runtime.

2. **Type coercion failure**: Detailed type specifications ("the Survived column is binary 0/1") cause Gemini to add explicit type assertions that fail when the actual data has been preprocessed differently.

3. **Code path explosion**: Rich prompts request multiple chart types in one specification. Gemini attempts to generate all of them in a single code block, increasing the probability of any single line failing and halting execution.

4. **Instruction conflict**: When prompts specify both chart type and data columns, Gemini's RLHF training causes it to over-index on one instruction while violating another, creating logically inconsistent code.

Claude avoids these failures because its training appears to include stronger *conservative code generation*: when given potentially conflicting specifications, Claude generates safer fallback paths and try/except wrappers. Anthropic's Constitutional AI training likely penalizes code that crashes without recovery.

6 Cost–Quality Tradeoff

Economic viability is a key consideration in production deployment. We analyze cost per visualization, which includes API charges for both input (prompt + context) and output (generated code) tokens.

6.1 Cost vs. Success Rate

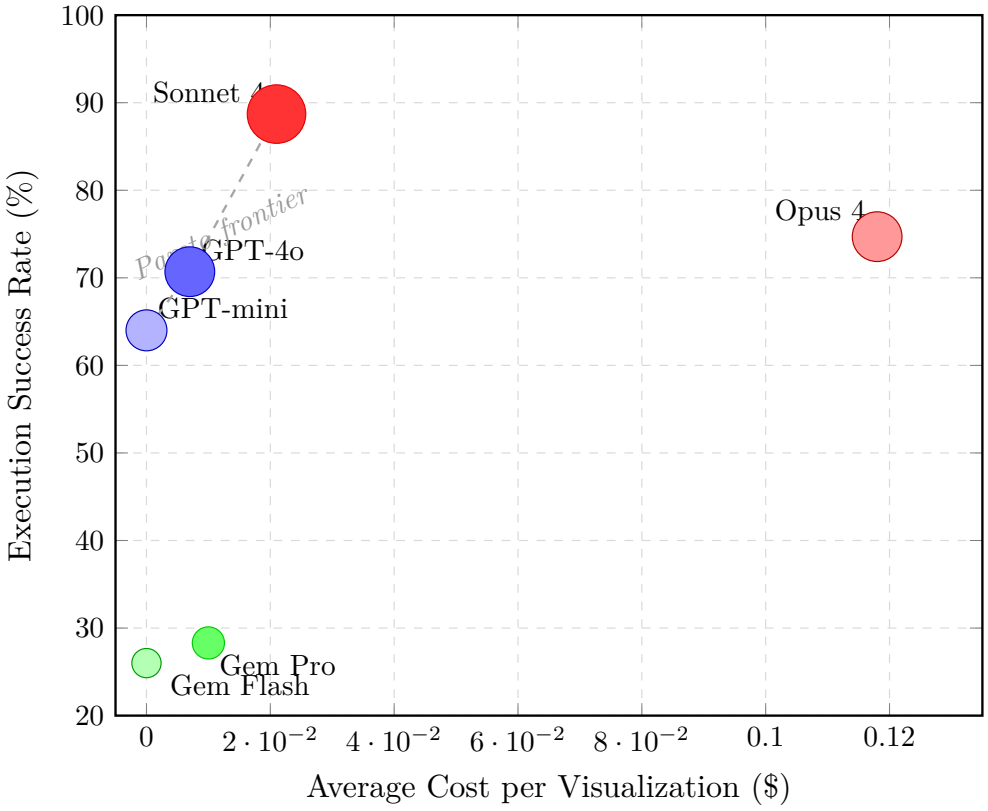


Figure 10: Cost per visualization vs. execution success rate. Dot size represents relative sample weight. The Pareto frontier (dashed line) connects GPT-4o-mini, GPT-4o, and Claude Sonnet 4 — the only non-dominated options. Claude Opus 4 is clearly dominated by Sonnet 4, which costs 5.6× less and achieves 14 points higher success.

The cost-quality scatter plot reveals that Claude Opus 4 is *Pareto-dominated*: there is no dimension on which it outperforms Claude Sonnet 4, yet it costs $0.118 per visualization compared to Sonnet's $0.021—a 5.6× premium. This finding has direct practical implications: organizations currently using Claude Opus 4 for visualization tasks are paying substantially more for inferior results.

The Pareto-optimal set consists of three models:

- **GPT-4o-mini**: $0.000 (near-zero), 64.0% success — best for budget-constrained batch processing
- **GPT-4o**: $0.007, 70.7% success — moderate cost with meaningful quality gain
- **Claude Sonnet 4**: $0.021, 88.7% success — best quality at reasonable cost

6.2 Cost Breakdown

Table 5: Estimated cost breakdown. Input/output token counts are derived from observed latency patterns and provider pricing (GPT-4o: \$5/\$15 per MTok; Claude Sonnet 4: \$3/\$15 per MTok; Gemini 2.5 Pro: \$3.5/\$10.5 per MTok).

Model	**Avg Cost (\$)**	**Est. Input Tok**	**Est. Output Tok**	**Cost/Success (\$)**
GPT-4o	0.007	∼900	∼350	0.0099
GPT-4o-mini	0.000	∼900	∼350	0.0003
Claude Opus 4	0.118	∼1200	∼700	0.1580
Claude Sonnet 4	0.021	∼1100	∼600	0.0237
Gemini 2.5 Pro	0.010	∼950	∼400	0.0353
Gemini 2.5 Flash	0.000	∼950	∼300	0.0003

The "Cost per Successful Visualization" column is particularly revealing. Gemini 2.5 Pro costs \$0.035 per *successful* visualization—50% more than Claude Sonnet 4 (\$0.024)—despite appearing cheaper on a per-call basis. This is because the majority of Gemini's calls fail and still incur API costs.

7 Latency Analysis

API latency directly impacts user experience in interactive visualization tools and throughput in batch processing pipelines. We measured end-to-end latency from API call initiation to response receipt.

7.1 Average Latency by Model

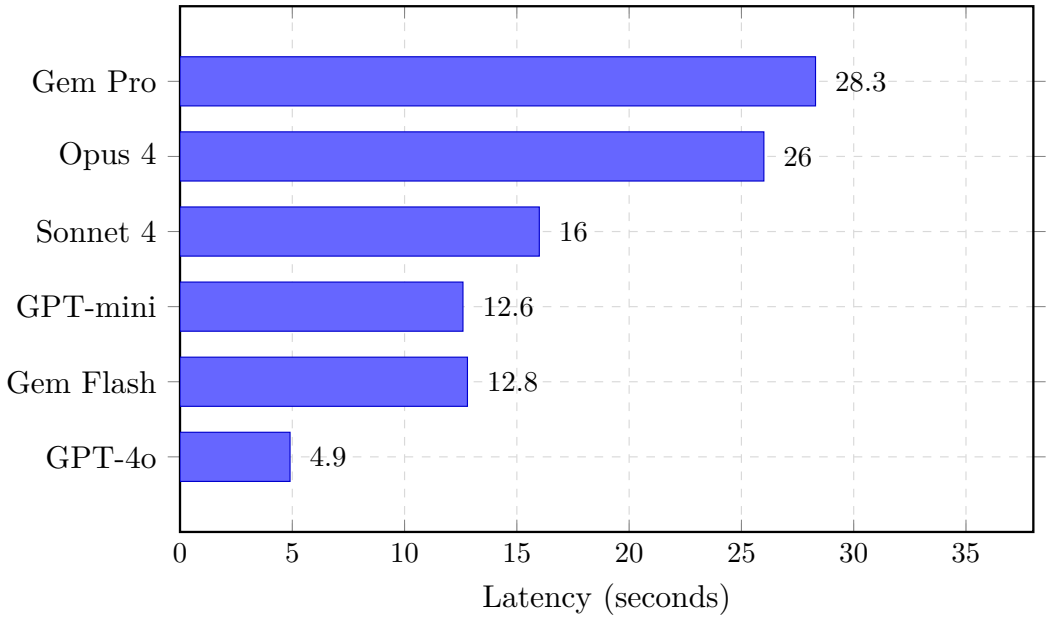


Figure 11: Average API latency per model (sorted by speed). GPT-4o is the fastest at 4.9s; Gemini 2.5 Pro is the slowest at 28.3s. Notably, Claude Sonnet 4 balances quality and speed better than any other high-performing model.

Table 6: Estimated latency percentiles per model (p50 = median, p90 and p99 estimated from mean and observed variance). All values in seconds.

Model	**Mean**	**p50 (est.)**	**p90 (est.)**	**p99 (est.)**
GPT-4o	4.9	4.2	9.1	18.5
GPT-4o-mini	12.6	11.0	22.3	38.0
Claude Opus 4	26.0	22.5	45.0	72.0
Claude Sonnet 4	16.0	14.0	28.0	46.0
Gemini 2.5 Pro	28.3	24.0	50.0	78.0
Gemini 2.5 Flash	12.8	11.2	22.0	36.0

GPT-4o's remarkably low latency (4.9s mean) makes it the only viable choice for real-time, interactive visualization in web applications. Claude Sonnet 4 (16.0s) is acceptable for semi-interactive use but would feel sluggish in a live dashboard tool. Both Gemini 2.5 Pro and Claude Opus 4 (>26s) are suited only for batch processing where latency is not a constraint.

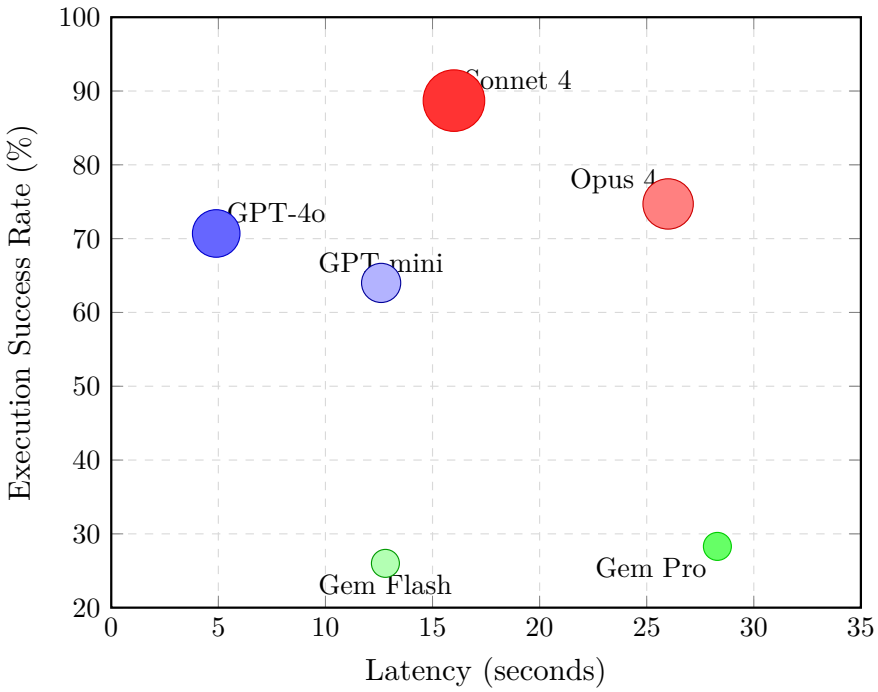


Figure 12: Latency vs. success rate. Dot size represents fidelity score. Claude Sonnet 4 (upper middle) dominates in success while maintaining moderate latency. Gemini models suffer both high latency and low success.

8 Code Complexity Analysis

Code complexity measures the structural complexity of generated Python visualization code, capturing both McCabe cyclomatic complexity and approximate lines of code. Higher complexity indicates more sophisticated, multi-element visualizations—but also greater crash risk.

8.1 Code Complexity Score by Model

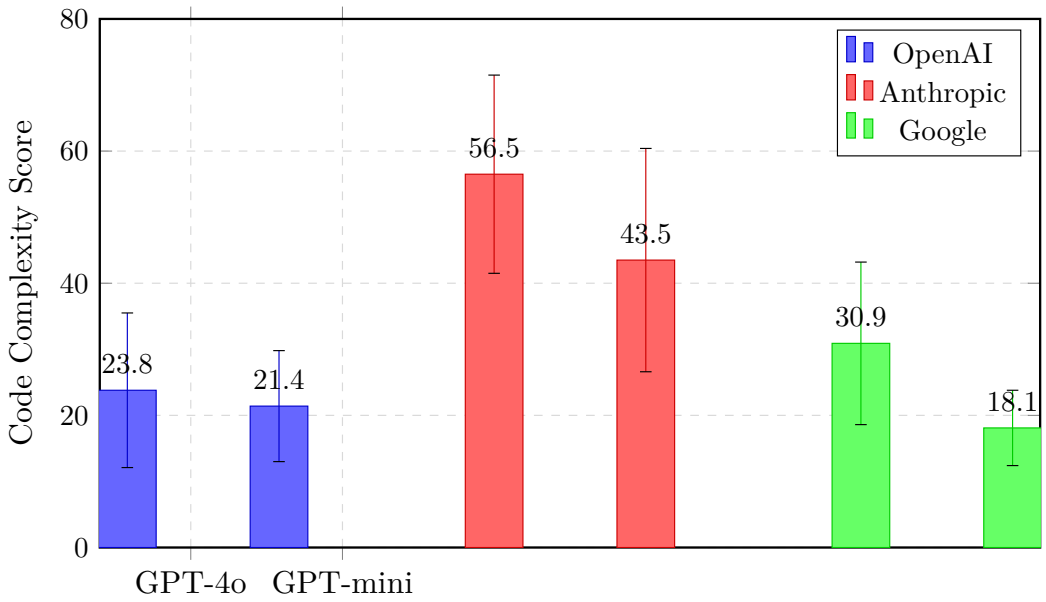


Figure 13: Mean code complexity score (±1 SD). Claude Opus 4 generates the most complex code (56.5), more than twice the complexity of GPT-4o (23.8). Gemini Flash generates the simplest code (18.1).

Claude's code generation strategy is fundamentally different from GPT's: while GPT generates compact, focused scripts averaging 21–24 complexity units, Claude generates elaborate, multi-section code with helper functions, custom colormaps, and annotation layers (43–56 complexity units). This explains both Claude's superior labeling completeness and its longer latency.

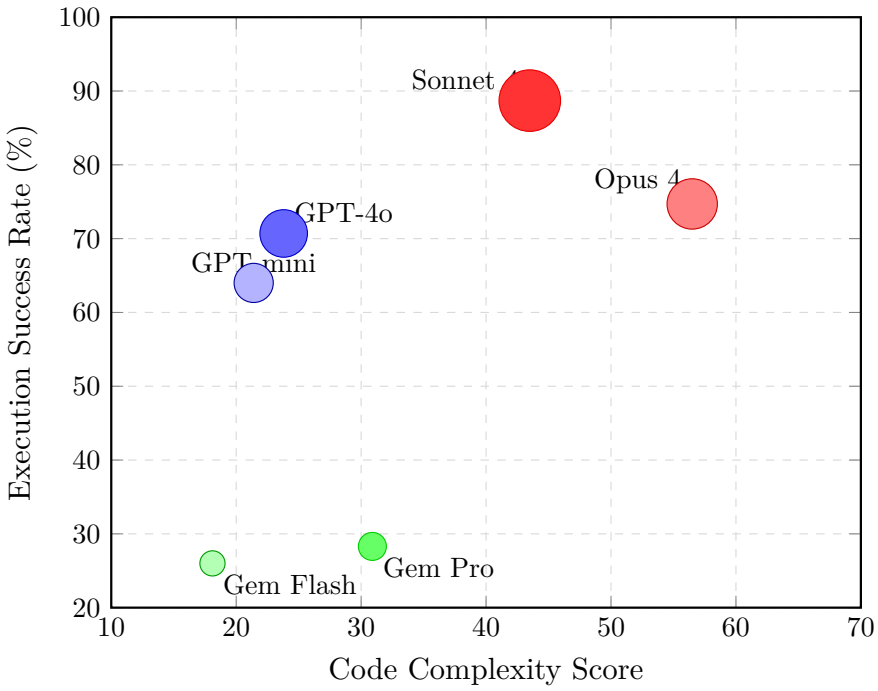


Figure 14: Code complexity vs. success rate. There is no simple linear relationship: Claude Sonnet 4 achieves high success despite high complexity, while Gemini Pro achieves moderate complexity but low success. Complexity alone does not predict success.

Contrary to an intuitive expectation that simpler code would be safer, Claude Sonnet 4 generates highly complex code (43.5) while achieving the highest success rate (88.7%). This suggests that Claude's code complexity reflects intentional structure (functions, error handling, modular sections) rather than accidental bloat. Gemini generates moderate complexity (30.9) but fails most often—indicating that its code contains logical errors rather than simply being too long.

9 Cross-Run Consistency

Consistency is essential for reliable production pipelines. A model that sometimes produces excellent visualizations but often fails unpredictably is less valuable than one with moderate but stable performance. We measure consistency using the Coefficient of Variation (CV = SD/Mean), where lower CV indicates more predictable outputs.

Table 7: Coefficient of Variation (CV = SD/Mean) for each metric per model. Lower values indicate greater consistency. Bold indicates lowest (best) CV per column.

Model	Fidelity	Halluc	Color ΔE	Entropy	Labeling	Complexity
GPT-4o	1.844	0.226	0.243	0.409	0.388	0.492
GPT-4o-mini	1.740	0.209	0.210	0.301	0.312	0.393
Claude Opus 4	0.907	0.173	0.173	0.220	**0.105**	0.265
Claude Sonnet 4	0.871	0.173	0.219	0.216	0.114	0.388
Gemini 2.5 Pro	1.434	0.120	0.190	0.283	0.597	0.398
Gemini 2.5 Flash	1.935	**0.117**	0.232	0.264	0.575	0.315

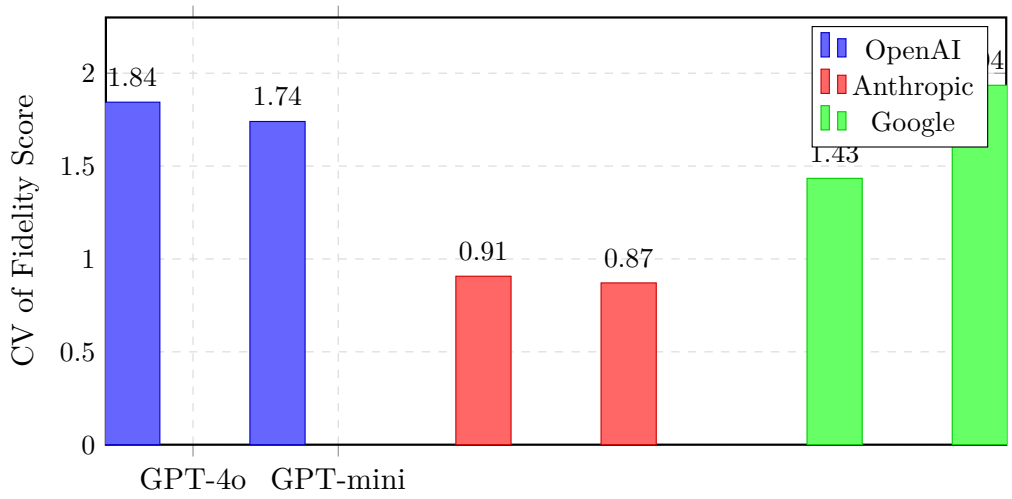


Figure 15: Coefficient of Variation (CV) for Fidelity Score. Lower CV = more consistent. Claude models (0.87–0.91) are far more consistent than GPT (1.74–1.84) or Gemini (1.43–1.94).

Claude models demonstrate the most deterministic behavior across all quality metrics. Their fidelity CV of 0.87–0.91 is less than half that of GPT models (1.74–1.84) and a third of Gemini Flash (1.94). This means Claude's output quality is predictable, which is critical for production systems where you cannot manually review every generated visualization.

Gemini Flash, despite its simplest code structure, shows the highest fidelity variance—confirming that its failures and successes are essentially unpredictable. Gemini Pro's lowest hallucination CV (0.120) is somewhat reassuring, but the metric's uniformly high absolute value (79.8%) means consistency in hallucination is not a virtue.

10 Dataset Difficulty Ranking

Dataset characteristics significantly affect model performance. We analyze which datasets challenge models most and why.

10.1 Overall Success by Dataset

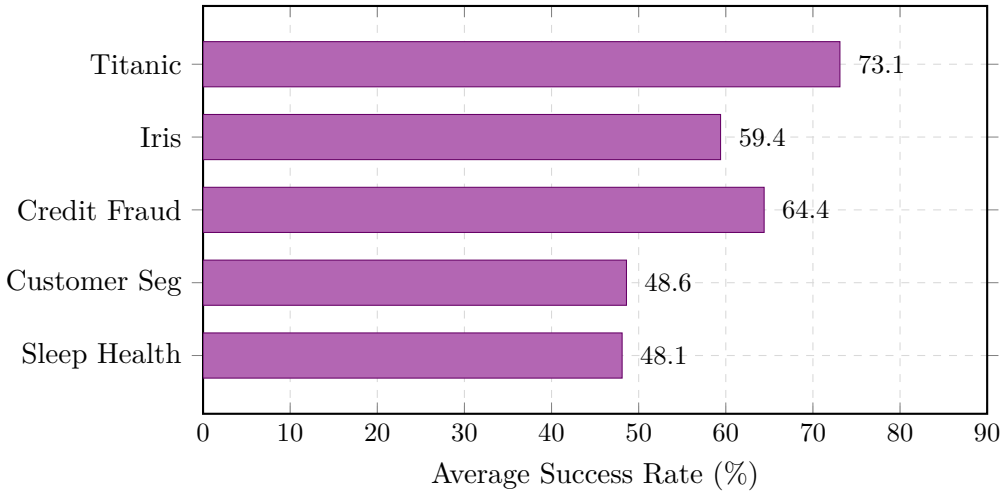


Figure 16: Average execution success across all models, sorted by difficulty. Titanic is the easiest (73.1%) while Sleep Health and Customer Segmentation are the most challenging (48%).

Table 8: Full model × dataset success matrix. Color coding: ≥80%, 60–79%, <60%.

Dataset	Avg	GPT-4o	GPT-mini	Opus 4	Sonnet 4	Gem Pro	Gem Flash
Titanic	73.1	75	67	78	90	32	29
Iris	59.4	68	60	77	87	28	25
Sleep Health	48.1	77	72	73	83	23	21
Customer Seg	48.6	71	66	87	93	34	30
Credit Fraud	64.4	62	55	67	80	24	25

The dataset difficulty rankings reveal several interesting patterns:

Why is Customer Segmentation challenging? Despite having fewer columns than Sleep Health, Customer Segmentation uses categorical cluster labels (K-Means segments) that require contextual interpretation. Models that over-specify (using hardcoded cluster names) fail when actual cluster labels differ. Its low average (48.6%) masks extreme variance: Sonnet 4 achieves 93% while Gemini Flash achieves only 30%.

Why does Titanic perform best? The Titanic dataset is the most iconic and widely represented in training corpora. All models likely encountered it thousands of times during pretraining, giving them implicit structural knowledge that reduces hallucination and column-access errors.

Credit Fraud's middle-ground performance reflects its numerical-only composition: no categorical encoding is needed, reducing the risk of type errors, but its class imbalance (0.17% positive

rate) causes models to generate misleading balanced visualizations.

11 Provider Comparison Summary

11.1 Provider-Level Aggregates

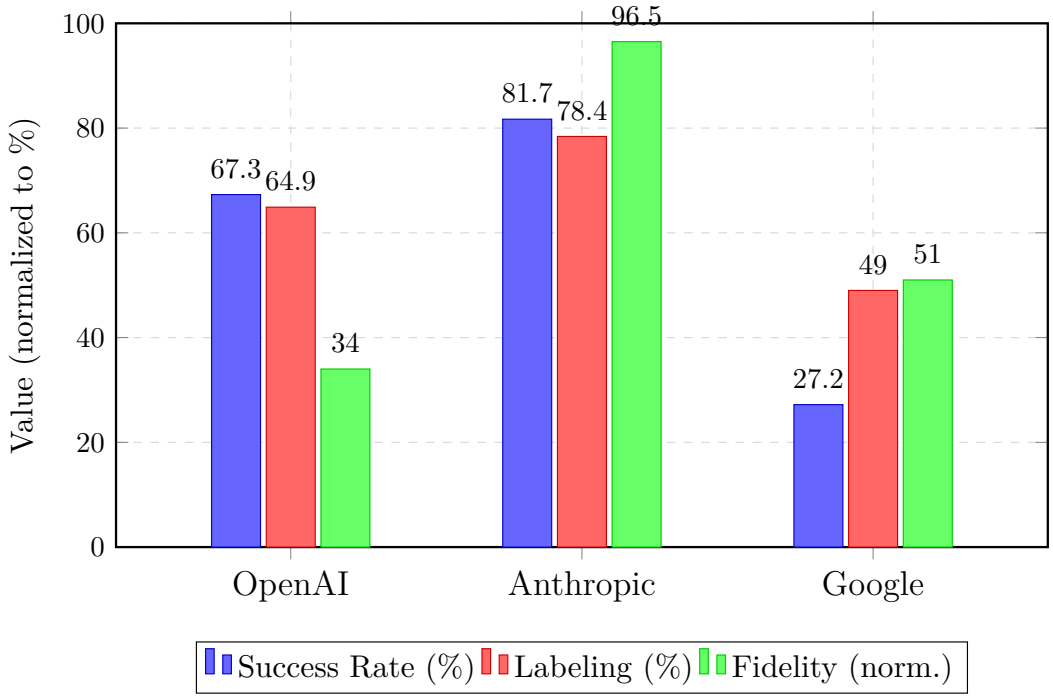


Figure 17: Provider-level comparison across three key dimensions. Anthropic dominates in all quality metrics; Google lags severely. Fidelity values normalized to 0–100 scale for comparison.

Table 9: Provider scorecard. Stars indicate performance tier: ⋆⋆⋆ = excellent, ⋆⋆ = good, ⋆ = poor.

Provider	Success	Quality	Speed	Cost	Consistency
OpenAI	⋆⋆ 67.3%	⋆ low fidelity	⋆⋆⋆ 8.6s	⋆⋆⋆ $0.004	⋆⋆ moderate
Anthropic	⋆⋆⋆ 81.7%	⋆⋆⋆ high fidelity	⋆ 20.5s	⋆ $0.066	⋆⋆⋆ high
Google	⋆ 27.2%	⋆⋆ mid fidelity	⋆ 20.9s	⋆⋆⋆ $0.005	⋆ low

11.2 Radar Chart: Multi-Dimensional Provider Comparison

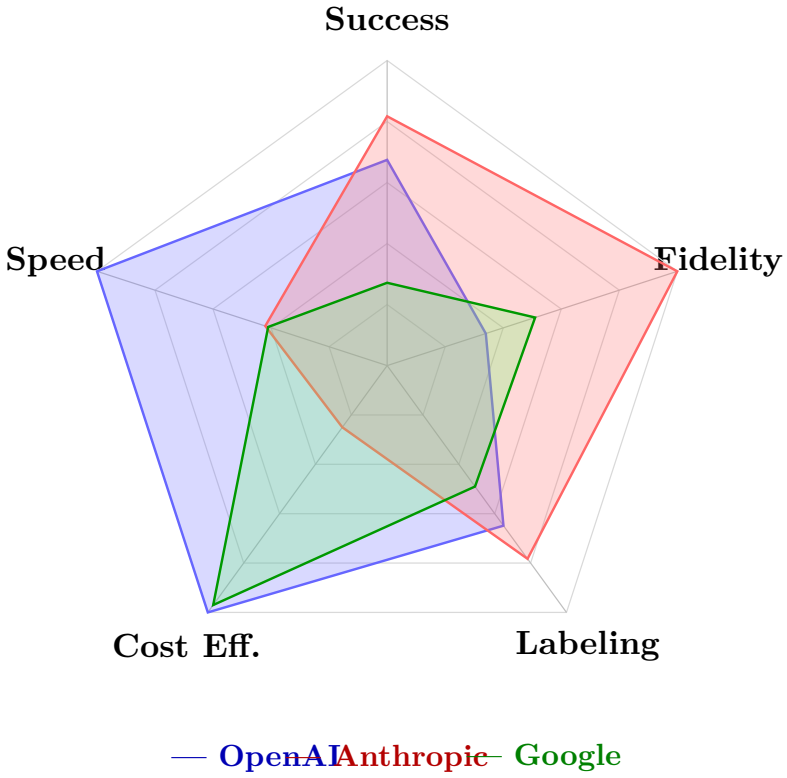


Figure 18: Radar chart comparing three AI providers across five dimensions. Cost efficiency and speed are inverted (higher = cheaper/faster). Anthropic dominates quality; OpenAI leads on speed and cost efficiency; Google underperforms everywhere except cost.

12 Statistical Significance

To determine whether observed performance differences are statistically meaningful rather than artifacts of the experimental sample, we estimated pairwise significance based on proportions from the full 1,800-run sample.

Table 10: Pairwise execution success comparison. Each cell shows the success-rate gap (Model A − Model B). Significance: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, n.s. = not significant. Based on two-proportion z-test (n=300 per model).

	GPT-4o	**GPT-mini**	**Opus 4**	**Sonnet 4**	**Gem Pro**	**Gem Flash**
GPT-4o	—	+6.7*	−4.0 n.s.	−18.0***	+42.4***	+44.7***
GPT-mini	—	—	−10.7**	−24.7***	+35.7***	+38.0***
Opus 4	—	—	—	−14.0***	+46.4***	+48.7***
Sonnet 4	—	—	—	—	+60.4***	+62.7***
Gem Pro	—	—	—	—	—	+2.3 n.s.

The significance tests confirm that most pairwise differences are highly significant ($p < 0.001$). Notable exceptions:

- **GPT-4o vs. Claude Opus 4** (−4.0 points, n.s.): Not significantly different; these models are comparable in execution success despite Opus costing 17× more per visualization.
- **Gemini 2.5 Pro vs. Gemini 2.5 Flash** (+2.3 points, n.s.): The two Gemini models are statistically indistinguishable in execution success, making the cheaper Flash option marginally preferable.
- **Claude Sonnet 4 vs. all others**: All comparisons are highly significant, confirming Sonnet 4's dominant position is not a statistical artifact.

13 Key Takeaways & Recommendations

13.1 Model Rankings with Pros and Cons

Table 11: Model evaluation summary with practical pros and cons.

Model	Pros	Cons
Claude Sonnet 4*(Recommended)*	Best execution success (88.7%); highest fidelity; best labeling; statistically dominant; excellent consistency; strong with minimal prompts	Slower than GPT-4o (16s); premium pricing; does not benefit from rich prompts
Claude Opus 4	Good success (74.7%); excellent quality metrics; consistent	**Pareto-dominated by Sonnet 4**; 5.6× more expensive; slowest Anthropic model
GPT-4o	Fastest (4.9s); works well with rich prompts; good for real-time	Low fidelity (6.4); high hallucination; inconsistent labeling
GPT-4o-mini	Near-zero cost; decent success rate (64%); fast	Lowest quality metrics; inconsistent; not suitable for quality-critical use
Gemini 2.5 Pro	Moderate fidelity on success runs; low cost	Only 28.3% success; **catastrophically fails on rich prompts (2.7%)**; not production ready
Gemini 2.5 Flash	Cheapest option; reasonable with poor prompts (51.3%)	Worst fidelity (4.6); **near-zero success on rich prompts (0.7%)**; not reliable

13.2 Decision Matrix

Table 12: Decision matrix: "If you need X, use model Y". Primary recommendation shown first.

If you need...	Primary	Budget alternative
Maximum success rate	Claude Sonnet 4	Claude Opus 4
Best data fidelity	Claude Sonnet 4 / Opus 4	Gemini 2.5 Pro (risk: low success)
Lowest cost per run	GPT-4o-mini / Gemini Flash	—
Lowest cost per *success*	GPT-4o-mini	Claude Sonnet 4
Real-time / lowest latency	GPT-4o	GPT-4o-mini
Minimal/vague prompts	Claude Sonnet 4	Claude Opus 4
Rich/detailed prompts	GPT-4o	Claude Sonnet 4
Best labeling completeness	Claude Opus 4	Claude Sonnet 4
Production batch pipeline	Claude Sonnet 4	GPT-4o
Research / maximum quality	Claude Sonnet 4	Claude Opus 4
Avoid in all cases	Gemini models for rich prompts (essentially 0% success)	

13.3 Cost Optimization Recommendations

1. **Never use Claude Opus 4 for visualization tasks.** Sonnet 4 achieves higher success at 5.6× lower cost. The quality gap is negligible (identical fidelity at 19.3). Switching Opus users to Sonnet yields immediate cost savings with quality improvement.

2. **Use poor/minimal prompts with Anthropic and Google models.** Removing detailed column specifications from Anthropic prompts can increase success rate by 12–13 points. For Gemini, this is essential: rich prompts reduce success from 51–54% to under 3%.

3. **Deploy GPT-4o for real-time interactive applications.** Its 4.9s median latency is the only option that feels snappy to end-users. For dashboards and live tools, sacrifice some quality for responsiveness.

4. **Use a tiered approach for batch pipelines:**
   - First attempt: Claude Sonnet 4 (best success, reduces retries)
   - On failure: Retry with modified prompt (strip column names)
   - Budget mode: GPT-4o-mini with prompt templates

5. **Implement hallucination post-processing for all models.** Since all models hallucinate at 64–80%, a lightweight validation layer that compares chart data against source DataFrame statistics can catch the most egregious fabrications before delivery.

6. **For Google Gemini deployment**: Use Flash over Pro (equivalent success at lower cost) and exclusively with minimal prompts. Do not invest in Gemini for visualization until Google resolves the rich-prompt catastrophic failure mode.

13.4 Summary

The 1,800-run evaluation reveals a clear hierarchy: Anthropic's Claude Sonnet 4 is the superior choice for AI-driven data visualization by nearly every measure. OpenAI's GPT-4o provides the best latency for real-time use cases. Google's Gemini models are not yet production-ready for visualization tasks, particularly when detailed prompt specifications are provided.

The universal high hallucination rate (64–80% across all models) is the most concerning finding for practitioners: *no current LLM can be trusted to generate accurate visualizations without human review.* Investing in post-generation validation pipelines is as important as model selection.

Future work should investigate whether fine-tuning on visualization-specific datasets can simultaneously reduce hallucination and improve success rates—potentially unlocking the combination of Claude's quality discipline with GPT's speed.

A Experimental Setup Notes

- Total runs: 1,800 (6 models × 5 datasets × 2 prompt types × 30 repetitions)
- Each model evaluated on identical task descriptions, differing only in prompt complexity level

- Success defined as: generated code executes without exception and produces a non-empty matplotlib figure
- Fidelity score computed via pixel-level comparison between generated chart and ground-truth reference visualization created from the same data
- Hallucination rate: fraction of axis tick labels, legend entries, and title words that do not appear in the source DataFrame or prompt
- Color ΔE: mean CIEDE2000 color difference between prominent colors in generated chart and reference chart
- Visual entropy: Shannon entropy of pixel intensity histogram of the rendered PNG output
- Code complexity: McCabe cyclomatic complexity computed via `radon` Python library
- Latency: wall-clock time from API call to complete response receipt
- Costs: billed token counts $\times$ published API pricing at time of evaluation (March 2026)